SOFTWARE REQUIREMENTS
SPECIFICATION

RefundPilot AI

AI-powered end-to-end refund resolution agent

Field	Value
Version	1.0
Status	Proposed implementation baseline
Prepared on	29 October 2026
Intended use	Development, testing, team coordination, hackathon planning
Notice	Independent project proposal; not an official PayPal product specification.

Confirm API availability and permissions against current PayPal documentation before implementation.

Document Control

Field	Value
Document	Software Requirements Specification — RefundPilot AI
Version	1.0
Status	Proposed implementation baseline
Prepared on	29 October 2026
Audience	Product team, developers, testers, reviewers, hackathon judges
Notice	Independent project proposal; not an official PayPal product specification. Verify API capabilities and permissions against current documentation.

Revision history: Version 1.0 defines the proposed MVP, system behavior, interfaces, data model, safety controls, testing approach, and deliverables.

1. Introduction

1.1 Purpose

This SRS defines the functional and non-functional requirements for RefundPilot AI, an AI-assisted web application that helps customers coordinate purchase-related refund resolution. A customer explains the problem in natural language; the system organizes the case, checks authorized payment/order information, gathers evidence, recommends an appropriate next action, and tracks verified progress.

1.2 Scope

RefundPilot is a case-management and workflow-orchestration layer. It is not a payment processor, merchant, legal authority, or substitute for PayPal's formal dispute decisions. The MVP supports complaint intake, case creation, purchase reference association, evidence collection, policy/status review, grounded AI explanation, guided merchant contact or escalation, refund tracking where supported, and case history.

1.3 Intended audience

- Product owner and hackathon team: scope, priorities, user journeys, acceptance criteria.
- Frontend developers: screens, states, accessibility, and client behavior.
- Backend developers: APIs, persistence, authorization, integrations, and orchestration.
- AI engineer: structured outputs, tools, grounding, safety, and evaluation.
- QA/reviewers: test cases, risks, and release evidence.

1.4 Definitions

Term	Definition
AI agent	Model-assisted component using controlled tools to interpret requests and propose permitted workflow steps.
Case	Persistent record of a refund-related issue and its history.
Merchant	Seller or business supplying the product/service.
PayPal Sandbox	PayPal test environment for supported developer payment flows.
Refund	Return of some or all of a captured payment, subject to transaction status, permissions, and API rules.
Dispute	Formal payment-provider process; AI can guide and organize evidence but does not decide the outcome.
Authoritative source	API or verified record that is the source of truth for a status/event.
PII	Personally identifiable information.
RBAC	Role-based access control.
Idempotency	Safe handling of repeated requests without unintended duplicate effects.

1.5 References

- PayPal Refund Captured Payment API: https://developer.paypal.com/docs/api/payments/v2/#captures_refund
- PayPal Orders API: https://developer.paypal.com/docs/api/orders/v2/
- PayPal Disputes API: https://developer.paypal.com/docs/api/customer-disputes/v1/
- PayPal Webhooks: https://developer.paypal.com/api/rest/webhooks/
- PayPal AI Hackathon: https://paypalaihackathon.devpost.com/
- Confirm current endpoint access, scopes, event coverage, eligibility, and hackathon rules before implementation.

2. Overall Description

2.1 Product perspective

Recommended stack: React + Vite + vanilla CSS; Node.js + Express; Prisma + PostgreSQL; an LLM API with structured outputs/tool calling; PayPal REST APIs and Sandbox; deployment on Vercel/Render or equivalent. Provider integrations must be behind server-side adapters.

2.2 Product goals

- Let customers explain refund issues in natural language.
- Show verified transaction facts and plain-language explanations.
- Help gather evidence and understand the next legitimate action.
- Maintain a traceable timeline so customers do not repeat the entire story.
- Demonstrate a meaningful PayPal sandbox workflow.
- Avoid unsupported claims, unauthorized financial actions, and refund guarantees.

2.3 Out of scope

- Guaranteeing merchant acceptance or faster settlement.
- Compelling a merchant to respond or issuing refunds without authority.
- Deciding formal disputes or promising outcomes.
- Accessing arbitrary private PayPal transactions without supported authorization.
- Live financial transactions in the hackathon demo; sandbox is the default.
- Full marketplace, call center, legal advice, or production-grade fraud adjudication.

2.4 User classes

User	Description	Needs/permissions
Customer	Person reporting damaged, defective, incorrect, not-as-described, or missing-delivery purchase.	Create/view own cases; upload evidence; approve actions; receive updates.
Demo user	Judge/tester using seeded sandbox cases.	Use visibly labeled demo data; no access to real customer information.
Merchant adapter	Participating merchant integration or simulated service.	Provide order/policy/response data within configured permissions.
Support operator (optional)	Authorized reviewer for ambiguous/failed cases.	Review assigned cases and audit history; cannot bypass financial controls.
Backend service account	Integration identity.	Scoped API access; credentials never exposed to browser.

2.5 Environment and constraints

- Modern desktop/mobile browser; HTTPS deployment; PostgreSQL database.
- PayPal capabilities depend on endpoint, account, transaction state, scopes, and current developer access.
- Merchant order/policy data requires an integration, customer-provided reference, or labeled demo fixture.
- LLM output may be wrong and must never be treated as authoritative payment state.
- Consequential actions require explicit user approval and server-side authorization.
- A merchant refund action, PayPal-recorded status, and availability of funds are distinct facts.

2.6 Dependencies

- PayPal developer account, sandbox buyer/seller accounts, REST credentials.
- LLM provider access and usage quota.
- Database and hosting availability.
- Merchant policy/order source or mock adapter.
- Optional notification service.
- Current API docs and a verified sandbox flow.

3. Functional Requirements

Priority: MUST = required for MVP; SHOULD = important if time permits; COULD = optional. Requirement IDs are stable for traceability.

3.1 Identity and access

ID	Priority	Requirement
FR-001	MUST	Provide a customer session/authentication mechanism.
FR-002	MUST	Allow users to access only their own cases or explicitly assigned demo cases.
FR-003	MUST	Validate authorization server-side on every case, evidence, and action request.
FR-004	MUST	Keep PayPal secrets, LLM keys, database credentials, and signing secrets server-side.
FR-005	SHOULD	Provide a demo mode with seeded identities and visibly labeled cases.

3.2 Complaint intake and case creation

ID	Priority	Requirement
FR-010	MUST	Accept a natural-language complaint.
FR-011	MUST	Support damaged, defective, wrong item, not-as-described, not received, refund pending, partial refund, and unclear categories.
FR-012	MUST	Extract candidate purchase reference, order date, amount, merchant, issue type, desired outcome, and dates when provided; absent values remain unknown.
FR-013	MUST	Ask clarifying questions if key information is missing or several purchases could match.
FR-014	MUST	Create a persistent case with unique ID and initial state.
FR-015	MUST	Let the customer review/correct extracted facts before consequential actions.
FR-016	SHOULD	Support selecting a transaction from an authorized list if the integration permits it.
FR-017	MUST	Never claim a purchase is verified until backend lookup succeeds.

3.3 Payment/order lookup

ID	Priority	Requirement
FR-020	MUST	Associate a case with a payment/order reference via authorized integration or clearly labeled fixture.
FR-021	MUST	Show source and retrieval time for payment facts.
FR-022	MUST	Distinguish original/captured/refunded/remaining amounts when available.
FR-023	MUST	Display source status as received, separately from the plain-language explanation.
FR-024	MUST	Handle missing, stale, unavailable, or contradictory data without inventing values.
FR-025	SHOULD	Refresh status via user action, polling, or configured webhook.
FR-026	MUST	Do not expose card details or unrelated account data to the UI or AI.

3.4 Evidence

ID	Priority	Requirement
FR-030	MUST	Allow evidence attachments such as product photos, order confirmation, delivery details, and merchant messages, within configured limits.
FR-031	MUST	Validate file type, size, and upload authorization server-side.
FR-032	MUST	Record evidence metadata, uploader, case, and upload time.
FR-033	MUST	Allow removal before external submission, subject to retention/audit constraints.
FR-034	SHOULD	Summarize evidence where feasible; never declare a product defective solely from uncertain visual AI assessment.
FR-035	MUST	Obtain explicit consent before sharing evidence externally.
FR-036	MUST	Treat uploaded files and extracted text as untrusted input.

3.5 Policy guidance

ID	Priority	Requirement
FR-040	MUST	Use configured merchant policy, customer-provided policy, or labeled demo policy.
FR-041	MUST	Identify source and effective/retrieval date where available.
FR-042	MUST	Describe policy interpretation as guidance, not guaranteed eligibility.
FR-043	MUST	If no reliable source exists, disclose this and do not fabricate terms.
FR-044	SHOULD	Explain conditions, evidence requirements, and return instructions in plain language.

3.6 AI diagnosis and resolution planning

ID	Priority	Requirement
FR-050	MUST	Classify the case and provide structured explanation grounded in supplied context.
FR-051	MUST	Separate verified facts, customer claims, AI inferences, and unknowns.
FR-052	MUST	Recommend only actions permitted by deterministic workflow rules.
FR-053	MUST	Never infer that refund is issued, merchant accepted liability, or dispute will be won without authoritative evidence.
FR-054	MUST	Identify missing evidence and ask concise follow-up questions when material.
FR-055	MUST	Explain the recommendation with references to available facts/policy.
FR-056	MUST	Handle low confidence/contradictions by clarifying or recommending human/support review.
FR-057	MUST	Defend against prompt injection in messages, files, merchant responses, and retrieved policy text.
FR-058	SHOULD	Maintain concise case context across turns.
FR-059	SHOULD	Record model/version and evaluation metadata without unnecessary sensitive content.

3.7 Merchant coordination

ID	Priority	Requirement
FR-060	MUST	Generate a structured merchant-contact summary with only necessary information.
FR-061	MUST	Never claim a message was sent unless the configured channel confirms successful submission.
FR-062	MUST	Without a real channel, provide a copyable draft or visibly simulated send action.
FR-063	MUST	Record merchant response only from a verified integration or labeled demo action.
FR-064	SHOULD	Support accepted, declined, more-information-requested, and no-response/unknown states.
FR-065	MUST	Do not impersonate a customer or merchant; obtain required approval for external messages.

3.8 Refund and dispute workflow

ID	Priority	Requirement
FR-070	MUST	Distinguish merchant-contact, refund-monitoring, and escalation-guidance stages.
FR-071	MUST	Use documented PayPal operation and required identifiers for sandbox refund; never fake an API success.
FR-072	MUST	Before a consequential action, validate transaction eligibility, amount, currency, permissions, and user confirmation.
FR-073	MUST	Prevent duplicate refund submissions with idempotency/retry controls where supported and local operation state.
FR-074	MUST	Distinguish submitted refund request from confirmed source status.
FR-075	MUST	Guide to the applicable PayPal dispute process; claim submission only if an authorized workflow confirms it.
FR-076	MUST	Never promise PayPal will compel a merchant or decide a dispute in the customer's favor.
FR-077	SHOULD	Support partial refunds where source data and permissions permit.
FR-078	MUST	Clearly label simulated merchant responses and demo refund outcomes.

3.9 Timeline and notifications

ID	Priority	Requirement
FR-080	MUST	Maintain chronological events for case creation, uploads, status changes, confirmations, API operations, and verified responses.
FR-081	MUST	Include event type, timestamp, actor/source, and reference when available.
FR-082	MUST	Show latest known state and stale/refresh failure indicators.
FR-083	SHOULD	Notify customers of material changes subject to preferences and available service.
FR-084	MUST	Avoid duplicate notifications for the same verified event.
FR-085	MUST	Allow follow-up/reopen when unresolved under case policy.

3.10 Support summary and closure

ID	Priority	Requirement
FR-090	MUST	Generate support-ready summary of complaint, purchase reference, verified facts, evidence, prior actions, and open questions.
FR-091	MUST	Distinguish customer assertions from verified records.
FR-092	MUST	Mark resolved only when closure condition is met; user closure must not imply refund completion.
FR-093	MUST	Retain history according to configured policy.
FR-094	SHOULD	Allow export/copy of support summary.

3.11 Administration and observability

ID	Priority	Requirement
FR-100	SHOULD	Allow authorized operators to view assigned cases and integration errors.
FR-101	MUST	Log security-relevant events and external action outcomes without secrets.
FR-102	MUST	Provide understandable errors for AI, provider, database, and upload failures.
FR-103	SHOULD	Expose basic case counts, provider error rates, and AI classification metrics.

4. Use Cases

UC-01: Create refund case

Field	Description
Actor	Customer
Preconditions	Valid session or explicitly labeled demo identity.
Main flow	1. Customer submits complaint. 2. AI extracts candidate facts. 3. System asks for missing information. 4. Customer reviews details. 5. Backend creates case and event. 6. UI shows case ID and next step.
Alternates	Ambiguous purchase: ask customer to choose. AI unavailable: manual fallback. Invalid input: explain required fields.
Postcondition	Persistent case with initial state and original complaint.

UC-02: Gather and submit evidence

Field	Description
Actor	Customer
Preconditions	Case exists and is accessible.
Main flow	Show evidence checklist; customer uploads file; server validates it; metadata is stored; customer reviews packet; explicit approval is obtained before external sharing; submission result is recorded.
Alternates	Reject invalid file; allow retry after upload failure; keep evidence private if customer declines sharing.
Postcondition	Evidence linked to case and any external sharing is auditable.

UC-03: Diagnose refund status

Field	Description
Actor	Customer
Preconditions	Authorized lookup available or labeled demo fixture selected.
Main flow	Backend retrieves source data; records retrieval time; rules classify state; AI generates grounded explanation; UI displays source status, timeline, uncertainty, and next action.
Alternates	API unavailable: show last verified data as stale. Missing reference: ask. Contradictory data: avoid definitive claim.
Postcondition	Traceable explanation based on available evidence.

UC-04: Initiate sandbox refund

Field	Description
Actor	Authorized demo user; backend performs provider call.
Preconditions	Sandbox capture exists, API supports operation, amount/currency valid, permissions configured, user confirms.
Main flow	Show transaction and amount; customer confirms; backend revalidates; submit supported request; store response/reference; show actual state; refresh later.
Alternates	API rejection: safe error. Timeout: mark outcome unknown and reconcile before retry. Duplicate click: block/reuse operation record.
Postcondition	Accurate operation record; no unverified success claim.

UC-05: Guide dispute escalation

Field	Description
Actor	Customer
Preconditions	Merchant resolution failed or customer requests escalation.
Main flow	Explain documented process; check evidence checklist; customer reviews summary; direct to official flow or invoke authorized API if available; record submission only when verified.
Alternates	Eligibility unclear: disclose uncertainty. API unavailable: provide guidance and summary, not false confirmation.
Postcondition	Customer has clear next step; outcome is not guaranteed.

UC-06: Continue unresolved case

Field	Description
Actor	Customer/system
Main flow	Check latest known state; identify permitted next action; notify/prompt as appropriate; append new events; keep case open until closure condition.
Postcondition	Case context and action history remain available.

5. External Interface Requirements

5.1 User interface

- Responsive desktop/mobile design; dashboard, New Case, My Cases, case detail, settings/help.
- Case detail shows complaint, transaction facts, evidence, timeline, next action, and state.
- Verified facts, customer claims, AI interpretations, and simulated data must look distinct.
- Require review/confirmation before consequential actions.
- Design loading, empty, stale, partial-success, success, and error states.
- Do not communicate status by color alone; include text/icons.
- Aim for WCAG 2.1 AA-aligned contrast, keyboard navigation, focus, and labels.

5.2 External software interfaces

Interface	Purpose	Constraints
PayPal REST API	Supported sandbox order/capture/refund/payment status operations.	Use current docs; server-side credentials; handle permissions/errors.
PayPal Disputes/official flow	Guide or retrieve dispute information where available.	Do not assume all endpoints/actions are available to every app/account.
LLM API	Structured extraction, grounded explanation, plan wording, summary.	Server-side; validate schema; model cannot authorize payment action.
PostgreSQL	Cases, events, evidence metadata, actions, audit.	Migrations, constraints, least privilege.
Private object storage	Evidence files.	Private by default, signed short-lived access, retention.
Merchant adapter/mock	Order/policy/response data for demo.	Label simulation; do not imply PayPal itself provides merchant communication.

5.3 Communication

- Use HTTPS/TLS for deployed traffic.
- Verify webhook signatures using documented provider procedure.
- Validate content type, request size, schema, authentication, and authorization.
- Bound retries and prevent duplicate side effects.

6. Data Requirements and Database Design

6.1 Conceptual entities

Entity	Purpose / illustrative fields
User	id, providerSubject/email, role, status, createdAt
RefundCase	id, userId, complaintText, issueType, requestedOutcome, state, createdAt, updatedAt
PurchaseReference	id, caseId, provider, orderId, captureId, merchantReference, sourceType, retrievedAt
PaymentSnapshot	id, caseId, currency, capturedAmount, refundedAmount, status, payloadHash, retrievedAt
EvidenceItem	id, caseId, uploaderId, fileKey, mimeType, size, evidenceType, sharingStatus, createdAt
CaseEvent	id, caseId, eventType, actorType, source, summary, occurredAt, metadata
ResolutionAction	id, caseId, actionType, status, proposedBy, confirmedAt, externalReference, idempotencyKey
MerchantInteraction	id, caseId, channel, status, messageReference, responseType, sourceType, createdAt
AIAnalysis	id, caseId, model, schemaVersion, issueClassification, missingFacts, recommendedAction, confidenceBand, createdAt
AuditLog	id, actorId, action, resourceType/id, outcome, requestId, timestamp
Notification	id, userId, caseId, eventKey, channel, status, sentAt

6.2 Integrity rules

- Each case has unique immutable ID.
- External identifiers include provider/type context.
- Amounts use decimal-safe values or integer minor units and explicit currency.
- Source timestamps are distinct from system timestamps.
- Refund actions retain external references and deduplication keys where supported.
- Evidence access is scoped to authorized users/cases.
- Audit events are append-only for normal application roles.
- AI output never overwrites authoritative payment fields.

6.3 Retention

Define retention periods before collecting real customer data. Use synthetic/sandbox data for the prototype. Apply deletion and retention rules consistently with applicable obligations and the published privacy notice; avoid collecting unnecessary data.

7. AI Agent Requirements and Guardrails

7.1 Agent responsibilities

- Extract complaint facts and ask clarifying questions.
- Summarize trusted payment/policy evidence supplied by backend tools.
- Identify missing evidence and generate a checklist.
- Recommend an action from a predefined permitted set.
- Draft merchant messages and support summaries for customer review.
- Explain state and uncertainty in plain language.
- Maintain concise case context across turns.

7.2 Agent prohibitions

- Never invent payment state, refund IDs, policy terms, deadlines, merchant responses, or settlement dates.
- Never trigger financial mutation solely from model output.
- Never decide a formal dispute or guarantee the outcome.
- Never follow instructions embedded in user/merchant/uploaded/retrieved text that override system rules.
- Never expose another user's data or internal credentials.
- Never present simulated actions as real-world actions.

7.3 Tool architecture

Use constrained orchestration. The LLM proposes a schema-validated action; backend policy logic validates state, permissions, transaction facts, and eligibility. Read-only tools may run within limits. Consequential tools require an expiring confirmation token bound to the reviewed action.

Tool	Access	Control
get_case_context	Read-only	Authenticated owner/assigned operator only.
lookup_payment_status	Read-only	Provider adapter validates reference and permissions.
get_policy_excerpt	Read-only	Configured policy source with metadata.
store_evidence_metadata	Write	Owner check and file validation.
draft_merchant_message	Draft	Customer reviews before sending.
send_merchant_message	External side effect	Only with configured channel, explicit approval, and delivery result.
initiate_sandbox_refund	Financial side effect	Sandbox default; revalidate capture, amount, permissions, confirmation, deduplication.
create_support_summary	Write	Authorized case access; redact unnecessary data.

7.4 Structured AI output

Suggested schema: issueType; customerGoal; verifiedFacts[]; customerClaims[]; missingInformation[]; policyFindings[]; observedPaymentState; recommendedAction; rationale; confidenceBand; userConfirmationRequired; safetyFlags[]; sourceReferences[]. Reject invalid outputs and fall back to safe clarification/guidance.

7.5 AI evaluation

- Use at least 20–30 synthetic cases across normal, partial, ambiguous, contradictory, and malicious inputs.

- Measure classification and next-action accuracy, unsupported-claim rate, missing-evidence detection, and uncertainty handling.
- Test prompt injection in complaint text, attachments, and merchant responses.
- Do not use production evidence for model training without appropriate legal basis and consent.

8. Non-Functional Requirements

ID	Category	Requirement
NFR-001	Performance	Ordinary non-provider UI/API actions should respond within 2 seconds at p95 under defined test load; long actions show progress.
NFR-002	Availability	Demo should be usable during judging and gracefully handle provider outage.
NFR-003	Security	Secrets stay out of source control, client bundles, and logs.
NFR-004	Privacy	Minimize data; use sandbox/synthetic demo data; restrict case/evidence access.
NFR-005	Authorization	Every object read/mutation is checked server-side.
NFR-006	Reliability	Never report consequential action success without verified service response.
NFR-007	Idempotency	Retries do not cause duplicate financial operations where provider/local controls permit.
NFR-008	Auditability	External actions, confirmations, refreshes, and state changes are traceable.
NFR-009	Usability	First-time user can understand case state and next step without technical help.
NFR-010	Accessibility	Keyboard navigation, focus, labels, contrast, and non-color status cues.
NFR-011	Maintainability	Separate provider adapters, rules, AI orchestration, routes, and persistence.
NFR-012	Portability	Documented local setup and environment-specific deployment config.
NFR-013	Scalability	Concurrent demo users cannot see each other's cases; avoid in-memory-only case state.
NFR-014	Observability	Use correlation IDs and safe logs without secrets or unnecessary personal data.
NFR-015	AI safety	Validate model output; backend retains authority over payment state and permissions.
NFR-016	Data freshness	Show retrieval time and stale-data indicators.
NFR-017	Compatibility	Support current major Chromium browsers and a current mobile browser.
NFR-018	Recovery	Upload/API failure allows safe retry without losing case description.

9. Security, Privacy, and Abuse Prevention

9.1 Authentication and authorization

- Use maintained auth library/provider; avoid custom password handling unless needed.
- Check owner/role on every case, evidence, timeline, and action endpoint.
- Use least-privilege database, PayPal, storage, and LLM credentials.
- Use secure session cookie attributes where applicable.
- Rate-limit login, case creation, uploads, AI calls, and external actions.

9.2 Payment-action safety

- Default to Sandbox.
- Separate read-only lookup from financial mutation endpoints.
- Show transaction, amount, currency, and action before confirmation.
- Revalidate immediately before execution; record confirmation and provider response.
- On timeout, mark outcome unknown and reconcile before retry.
- Never store raw card number, CVV, or unnecessary payment credentials.

9.3 Evidence and AI safety

- Validate file type/signature and size; use private storage and short-lived signed URLs.
- Consider malware scanning for production.
- Do not send evidence to model/merchant without purpose and required consent.
- Treat external text as untrusted data; use tool allowlists and argument validation.
- Model output cannot set identity, amount, permissions, or case ownership.
- Redact secrets and unnecessary personal data from prompts/logs.

9.4 Transparency

- Tell users when data is sent to an AI provider or external service.

- Clearly label sandbox, simulated, and verified live-source information.
- Provide a privacy notice for hosted demo.
- Do not claim regulatory compliance or security certification without assessment.

10. State Model and Business Rules

10.1 Proposed states

State	Meaning	Example transitions
NEW	Case created; facts not reviewed.	NEEDS_INFO, INVESTIGATING, CLOSED_BY_USER
NEEDS_INFO	Material information/evidence missing.	INVESTIGATING, CLOSED_BY_USER
INVESTIGATING	Checking records/policy.	MERCHANT_ACTION_REQUIRED, REFUND_MONITORING, ESCALATION_GUIDANCE, NEEDS_INFO, ERROR_REVIEW
MERCHANT_ACTION_REQUIRED	Merchant contact/request needed or awaiting response.	MERCHANT_RESPONSE_RECEIVED, ESCALATION_GUIDANCE, NEEDS_INFO
MERCHANT_RESPONSE_RECEIVED	Verified or labeled simulated response recorded.	REFUND_MONITORING, NEEDS_INFO, ESCALATION_GUIDANCE, RESOLVED
REFUND_MONITORING	Refund initiated or source says processing.	RESOLVED, ERROR_REVIEW, ESCALATION_GUIDANCE
ESCALATION_GUIDANCE	Guidance through supported formal process.	INVESTIGATING, RESOLVED, ERROR_REVIEW
ERROR_REVIEW	Conflict, integration failure, or uncertain outcome.	INVESTIGATING, NEEDS_INFO, ESCALATION_GUIDANCE
RESOLVED	Defined closure condition met and recorded.	REOPENED if allowed
REOPENED	Customer reopens case.	INVESTIGATING, NEEDS_INFO
CLOSED_BY_USER	User closed case; does not imply refund occurred.	REOPENED if allowed

10.2 Business rules

- BR-01: Complaint is not proof merchant/provider accepted responsibility.
- BR-02: Only authoritative data establishes recorded payment/refund status.
- BR-03: AI text alone never means refund completed.
- BR-04: User closure is distinct from resolution.
- BR-05: Policy interpretation cites a source or discloses absence of one.
- BR-06: Every external action requires authorization, validation, and audit.
- BR-07: Merchant responses cannot be invented; simulated ones are labeled.
- BR-08: Validate amount/currency and transaction before refund request.
- BR-09: Reconcile ambiguous timeouts before retrying side effects.
- BR-10: AI cannot override state transitions/access controls.
- BR-11: Keep verified facts, customer assertions, and AI inference distinct.
- BR-12: Do not promise arrival dates unless an authoritative source provides them and context is shown.

11. Proposed Internal REST API

These are proposed RefundPilot application endpoints, not PayPal endpoints. Names/payloads may change. Authenticate and authorize all endpoints unless explicitly public.

Method / endpoint	Purpose and key behavior
POST /api/cases	Create case; accept complaint/reference; create initial event.
GET /api/cases	List only caller's cases; paginate/filter.
GET /api/cases/:caseId	Return summary, state, timeline, evidence metadata, next action.
PATCH /api/cases/:caseId	Update permitted fields; client cannot set verified payment status.
POST /api/cases/:caseId/analyze	Fetch trusted context, run AI, validate schema, apply policy.
POST /api/cases/:caseId/evidence	Validate ownership/file and store evidence.
DELETE /api/cases/:caseId/evidence/:evidenceId	Delete only permitted unsubmitted evidence.
POST /api/cases/:caseId/merchant-message/draft	Draft message; no external send.
POST /api/cases/:caseId/merchant-message/send	Only with configured channel, explicit approval, and delivery result.

Method / endpoint	Purpose and key behavior
POST /api/cases/:caseId/actions/:actionId/confirm	Validate confirmation token, action hash, user, state, and expiry.
POST /api/cases/:caseId/refunds	Sandbox-first refund; validate transaction/permission; audit and deduplicate.
POST /api/cases/:caseId/refresh	Refresh external state and append event.
GET /api/cases/:caseId/timeline	Return chronological authorized events.
GET /api/health	Safe health status without secrets.

11.1 API conventions

- JSON request/response except multipart uploads.
- Consistent error object: code, user-safe message, correlation ID.
- Use appropriate 400/401/403/404/409/422/429/5xx responses.
- Validate all payloads server-side; limit request/file sizes.
- Use pagination and idempotency for side-effect operations where supported.
- Never return raw provider credentials, internal stack traces, or unfiltered provider payloads.

12. Error Handling and Recovery

Failure	Required behavior
LLM timeout/invalid output	Preserve case; fallback to manual questions or deterministic guidance.
Provider auth/permission failure	Do not retry endlessly; show verification unavailable; never claim success.
Provider timeout after refund request	Mark outcome unknown; reconcile before retry.
Database unavailable	Return safe error; do not claim case/action was saved.
Evidence upload failure	Preserve form/case; allow retry; do not mark uploaded until confirmed.
Merchant channel unavailable	Provide copyable draft or labeled simulation; never claim sent.
Stale provider data	Show last verified time and offer refresh.
Conflicting facts	Show conflict, ask for clarification or recommend review.
Unauthorized access	Reject without revealing another user's case exists.
Duplicate action	Return existing operation state or safely reject; no second financial side effect.

13. Testing and Verification

13.1 Test levels

- Unit: state transitions, amount validation, policy rules, schema validation, authorization, idempotency.
- Integration: database, PayPal sandbox adapter, file storage, notification service, LLM output validation.
- End-to-end: complaint → evidence → diagnosis → approved action → timeline.
- Security: cross-user access, malicious upload, prompt injection, secret exposure, replayed confirmation, rate limiting.
- AI evaluation: classification, grounding, uncertainty, next-action accuracy, unsupported cases.
- Usability: first-time tester completes core flow without coaching.

13.2 Required scenarios

ID	Scenario	Expected result
TC-001	Damaged item, clear order reference	Create case, identify category, show evidence checklist.
TC-002	No purchase reference	Ask for reference; do not guess.
TC-003	Two matching purchases	Ask user to select correct purchase.
TC-004	Refund not initiated in source	Say no refund is visible; no false status.
TC-005	Refund pending	Show source state/retrieval time; no guaranteed date.
TC-006	Partial refund	Distinguish original, refunded, remaining amounts.
TC-007	Merchant declines	Record source; offer permitted next steps.
TC-008	No merchant channel	Copyable draft; no false sent confirmation.
TC-009	Refund API error	Safe error; no false success.
TC-010	Timeout after submit	Mark unknown and reconcile; prevent duplicate.
TC-011	Cross-user case access	Reject without data leakage.
TC-012	Prompt injection in upload	Ignore embedded instructions; enforce tool policy.
TC-013	Policy source unavailable	Disclose uncertainty; do not invent terms.
TC-014	AI service unavailable	Manual fallback; preserve case.
TC-015	Simulated merchant response	Visibly label as simulation.
TC-016	User closes unresolved case	Use CLOSED_BY_USER, not RESOLVED.

13.3 Proposed acceptance criteria

- At least 90% classification accuracy on a curated 20–30-case synthetic set, with actual results documented.
- 100% of demo transaction amounts/statuses/timestamps trace to API response or are labeled synthetic.
- No financial action executes solely from LLM output; backend validation and explicit confirmation are mandatory.
- Core demo completes in under two minutes under normal conditions.
- Required tests pass or known failures are transparently documented.
- Clean checkout can run using README and environment template.

These are targets, not measured results. Do not claim they are achieved until the test results are recorded.

14. Deployment and Operations

14.1 Environments

Environment	Purpose	Rules
Local	Development/tests	Local .env; no committed secrets; sandbox credentials.
Staging/demo	Judging/team integration	HTTPS; isolated DB; seeded data; visible simulation labels.
Production future	Real users/integrations	Requires privacy/security review, support, retention policy, provider access review, monitoring.

14.2 Environment variables

- DATABASE_URL — PostgreSQL connection string.
- PAYPAL_CLIENT_ID and PAYPAL_CLIENT_SECRET — server-side credentials; separate sandbox/live configuration.
- PAYPAL_BASE_URL — trusted configured base URL, never client-controlled.
- LLM_API_KEY and LLM_MODEL — server-side AI settings.
- APP_BASE_URL — trusted origin.
- SESSION_SECRET/auth-provider settings.
- Object-storage credentials/bucket if evidence uses external storage.
- Webhook verification configuration if webhooks are enabled.

14.3 Deployment checklist

- Run migrations; seed only non-sensitive fixtures.
- Configure sandbox credentials/accounts.
- Configure CORS, HTTPS, secure sessions, and rate limits.
- Verify webhook signatures if applicable.
- Check health endpoint, safe logs, and error handling.

- Run deployed smoke tests.
- Check secrets are absent from bundles, repo, logs, and demo video.
- Provide resettable demo path.

15. MVP Plan and Deliverables

15.1 MVP scope

- Complaint intake and case creation.
- Payment/order lookup via sandbox integration or labeled fixtures.
- Evidence upload and checklist.
- Deterministic state engine with limited refund states.
- Grounded AI explanation.
- Merchant draft and mock response adapter if no real channel exists.
- At least one working sandbox refund workflow where supported.
- Timeline, safe errors, tests, README, license, public repository, hosted demo, and short demo video.

15.2 Suggested phases

Phase	Work	Exit condition
1 Validate	Read API docs, configure sandbox, prove capture/refund flow, decide merchant adapter.	One documented sandbox flow succeeds.
2 Backend	Schema, case APIs, state machine, provider adapter, audit events.	Cases persist; transitions tested.
3 UI	Dashboard, complaint form, case details, evidence, timeline.	Non-AI journey works.
4 AI	Structured extraction, grounded explanation, safe action proposal, summaries.	AI passes synthetic tests and cannot bypass backend.
5 Integration	Sandbox refund, status refresh, merchant simulation, failures.	Repeatable end-to-end demo.
6 Submission	Tests, deployment, README, license, video, presentation.	All deliverables verified.

15.3 Deliverables

- Responsive frontend.
- Backend REST API and database schema/migrations.
- PayPal sandbox integration and setup guide.
- AI orchestration, prompt/schema versioning, synthetic evaluation set.
- Merchant adapter or clearly labeled simulation.
- Automated tests and documented results.
- README: architecture, setup, env vars, limitations, demo path.
- Public source repository and open-source license as required by event rules.
- Hosted demo and public video within current submission limits.

16. Risks, Open Questions, Future Enhancements

16.1 Risks and mitigations

Risk	Impact	Mitigation
No access to transaction history	Lookup flow blocked	Validate access early; use sandbox references and labeled fixtures.
No merchant communication API	Cannot coordinate with all sellers	Use authorized integration if available; otherwise copyable draft and demo adapter.
AI makes unsupported claims	Confusion/loss of trust	Grounded context, schema, deterministic rules, uncertainty labels.
Scope too broad	Incomplete demo	One damaged-item journey plus a few branches.
Duplicate refund after timeout	Potential financial harm	Sandbox-first, idempotency, reconciliation, confirmation, audit.
Evidence exposure	Privacy incident	Private storage, authorization, signed URLs, minimal retention.
Looks like generic chatbot	Weak differentiation	Show real state transitions, API results, evidence packet, continuity.

16.2 Open questions before implementation

- Will login, demo identity, or both be used?
- Which PayPal APIs/scopes are available to the developer account?
- Will merchant coordination use a real integration or simulation?
- Where will evidence be stored and what are file limits?
- Which LLM provider and structured-output mechanism will be used?
- Will MVP include email notifications or only in-app updates?
- What retention period and privacy notice apply to hosted demo?
- Which actions are genuinely API-executable versus guidance-only?

16.3 Future enhancements

- Merchant support dashboard and triage.
- Multilingual intake and explanation.
- Policy retrieval across merchants with source/version tracking.
- Webhook-driven updates where supported.
- Evidence extraction with human confirmation.
- Privacy-safe analytics on refund blockers and resolution times.
- Additional authorized merchant/order integrations.
- Human handoff with complete summary and audit history.

17. Traceability Matrix

Goal	Requirements	Verification
G1 Create case from natural language	FR-010–FR-017	UC-01; TC-001–003
G2 Show verified transaction/refund state	FR-020–FR-026	UC-03; TC-004–006
G3 Collect evidence safely	FR-030–FR-036	UC-02; upload/security tests
G4 Grounded AI guidance	FR-050–FR-059	AI evaluation; TC-012–014
G5 Merchant coordination	FR-060–FR-065	Merchant integration/mock tests; TC-008, 015
G6 Safe sandbox refund	FR-070–FR-078	UC-04; TC-009–010
G7 Track/close cases correctly	FR-080–FR-094	UC-06; TC-016
G8 Protect data/actions	FR-001–005; NFR-003–015	Security tests and audit review

Appendix A. Sample Case Record

Field	Illustrative value
Case ID	RP-DEMO-0001
Complaint	Headphones arrived with a cracked earcup; customer wants a refund.
Issue type	DAMAGED_ITEM
Purchase amount	INR 4,000.00 — demo value
Payment reference	Sandbox/demo reference only
Evidence	Two damage photos and order confirmation
Observed payment state	Captured — illustrative; replace with actual API response.
Refund state	Not initiated/not verified yet
Case state	INVESTIGATING
Recommended next step	Review merchant policy and prepare request; do not claim refund approval.
Data provenance	Example fixture, not a real transaction.

The example is illustrative only and must be visibly labeled as demo data.

Appendix B. Sample AI Output

Conceptual structured output; not an actual model result. Validate against the chosen schema.

```
{
"issueType": "DAMAGED_ITEM",
"customerGoal": "Request a refund",
"verifiedFacts": [{"field":"paymentStatus","value":"COMPLETED","source":"sandbox_api_response"}],
"customerClaims": [{"claim":"The item arrived damaged","source":"customer_message"}],
"missingInformation": ["Merchant return policy","Evidence submission confirmation"],
"recommendedAction": "PREPARE_MERCHANT_REQUEST",
"rationale": "A merchant request and evidence review are appropriate before any refund outcome is assumed.",
"confidenceBand": "MEDIUM",
"userConfirmationRequired": true,
"safetyFlags": ["REFUND_NOT_CONFIRMED"],
"sourceReferences": ["payment_snapshot_id","case_event_id"]
}
```

Backend must validate references and allowed action names; model confidence is not proof of correctness.

Appendix C. Definition of Done

- All MUST requirements implemented or explicitly deferred with reason.
- Damaged-product journey works from intake to supported next step.
- At least one sandbox workflow demonstrated with actual API responses, or limitations clearly disclosed.
- All simulated events and fixtures visibly labeled.
- No critical cross-user access, secret exposure, or unconfirmed financial action remains.
- AI evaluation set and actual results documented.
- Required tests pass or known failures listed.
- README, architecture diagram, environment template, license, repository, demo deployment, and video ready.
- Team can explain what AI, PayPal, merchant integration, and manual processes each do.